

\$author-display-name

08/20/2026



Vivado Design Suite User Guide: Using Constraints (UG903)

\$author-display-name

Introduction

Migrating From UCF Constraints to XDC Constraints

Navigating Content by Design Process

About XDC Constraints

Vivado Design Suite User Guide: Using Constraints (UG903)

Introduction

Migrating From UCF Constraints to XDC Constraints

The AMD Vivado™ Integrated Design Environment (IDE) uses Xilinx design constraints (XDC), and does not support the legacy user constraints file (UCF) format.

Understand that XDC constraints differ significantly from UCF constraints. XDC constraints follow the standard Synopsys design constraints (SDC) format. SDC has evolved for more than 20 years, making it the most widely used and proven format for describing design constraints.

 **Video:** For training on migrating UCF constraints to XDC, refer to the [Vivado Design Suite QuickTake Video: Migrating UCF Constraints to XDC](#).

If you already know UCF but are new to XDC, review Differences Between XDC and UCF Constraints in the ISE to Vivado Design Suite Migration Guide ([UG911](#)). This chapter also describes how to convert existing UCF files to XDC so you can use them as a starting point for creating XDC constraints.

 **Important:** XDC has key differences from UCF that you must understand to properly constrain your design. The UCF-to-XDC conversion utility does not replace your knowledge or ability to create XDC constraints. This guide describes each XDC constraint for your reference.

Navigating Content by Design Process

AMD Adaptive Computing documentation is organized around a set of standard design processes to help you find relevant content for your current development task. You can access the AMD Versal™ adaptive SoC design processes on the [Design Hubs](#) page. You can also use the [Design Flow Assistant](#) to better understand the design flows and find content that is specific to your intended design needs. This document covers the following design processes:

- **Hardware, IP, and Platform Development**

Creating the PL IP blocks for the hardware platform, creating PL kernels, functional simulation, and evaluating the AMD Vivado™ timing, resource use, and power closure. Also involves developing the hardware platform for system integration. Topics in this document that apply to this design process include:

- [Dedicated Hardware Resources](#)
- [IP and Sub-Module Constraining with XDC](#)

About XDC Constraints

XDCs combine the industry-standard SDCs version 1.9 with AMD proprietary physical constraints.

XDC constraints have these properties:

- They act as Tcl commands, not simple strings.
- The Vivado Tcl interpreter processes them like any other Tcl command.
- Vivado reads and parses them sequentially, the same as other Tcl commands.

Entering XDC Constraints

1. Decide how to provide constraints:
 - Store the constraints in one or more XDC files.
 - Generate the constraints with an unmanaged Tcl script.

2. Follow these steps to store constraints in XDC files:
 - a. Create or update the `.xdc` file.
 - XDC files only accept the `set`, `list`, and `expr` built-in Tcl commands.
 - Refer to [Supported XDC and SDC Commands](#) for the full list.
 - b. Load the `.xdc` file into memory using one of these methods:
 - Run the `read_xdc` command.
 - Add the file to one of your project constraints sets.
3. Follow these steps to generate constraints with an unmanaged Tcl script:
 - a. Create a `.tcl` script.
 - You can use any common Tcl command for selecting design objects and defining design constraints, including conditional and looping control structures.
 - b. Execute the script using one of these methods:
 - Run the `source` command.
 - In non-project mode, run `read_xdc -unmanaged`.
 - In project mode, do the following:
 - i. Add the Tcl script to a project constraints set.
 - ii. Add the file to the project.
 - iii. Set the file property to `Tcl`.

Following is an example:

```
add_files -fileset constrs_me timing.xdc
set_property file_type Tcl [get_files -of
[get_filesets constrs_me] timing.xdc]
```

★ **Tip:** Unlike XDC files, unmanaged Tcl scripts can use any Tcl command for selecting objects and defining constraints, including conditional or looping structures.

 **Important:** You can mix XDC files and Tcl scripts in the same constraints set. Keep these limits in mind:

- Vivado saves modified constraints back to their original location only if they originally came from an XDC file.
- The tool does not manage constraints from unmanaged Tcl scripts, and you cannot modify them interactively.

For more information, refer to [Constraints Methodology](#).

Note on `source` vs. `read_xdc`

For XDC constraints, `source` and `read_xdc` behave differently:

1. Constraints imported with `source` are not saved in the checkpoint in the same order as they are imported.
2. Constraints imported with `read_xdc` are saved first, then those imported with `source`.
3. To save all constraints in the same order as they are applied to the design, use `read_xdc -unmanaged` instead of `source`.

Validating Constraints

After you load your design in memory, use the Tcl Console and Vivado reports to analyze and debug timing and physical constraints.